MODULE 19 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Recursion and generators

Functions that call themselves, how to trace them, when they never stop, and functions that yield values one at a time.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 19 of 28

admin@sthithapragna.com

A recursive function calls itself on a smaller problem, and must reach a **base case** that stops. A generator **yields** values one at a time, only when asked.

A base case, and a call on a smaller problem

```
1  def fact(n):
2      if n <= 1:              # base case: answer directly
3          return 1
4      return n * fact(n - 1) # recursive case: a smaller problem
5
6  print(fact(4))             # 24
```

2-3 The base case needs no further call. Without it, the calls never end.

4 Each call passes a smaller n, moving towards the base case.

Calls stack up like plates, then unwind

CALL	IS WAITING TO WORK OUT	RETURNS
`fact(4)`	4 * fact(3)	4 * 6 = 24
`fact(3)`	3 * fact(2)	3 * 2 = 6
`fact(2)`	2 * fact(1)	2 * 1 = 2
`fact(1)`	base case	1

EXAM TECHNIQUE

Read the middle column downwards as the calls stack up, then the right column upwards as they return.

A wrong base case poisons every result

```
1  def total(x):
2      if x == 0:
3          return 0            # right for adding
4      return x + total(x - 1)
5  def product(x):
6      if x == 0:
7          return 0            # wrong for multiplying
8      return x * product(x - 1)
9  print(total(3), product(3))   # 6 0
```

3 3 + 2 + 1 + 0 is 6. Adding 0 at the end changes nothing.

7 3 * 2 * 1 * 0 is 0. Multiplying by 0 at the bottom wipes out everything.

Before the call, or after it

PRINT, THEN RECURSE

Counts down

```
1  def down(n):
2      if n == 0:
3          return
4      print(n, end=" ")
5      down(n - 1)
6  down(3)
```

3 2 1. Each call prints before going deeper.

RECURSE, THEN PRINT

Counts up

```
1  def up(n):
2      if n == 0:
3          return
4      up(n - 1)
5      print(n, end=" ")
6  up(3)
```

1 2 3. Each call prints only after the deeper calls finish.

THE TIEBREAKER

Work done before the recursive call happens on the way down. Work after it happens on the way back up.

No base case, or one that is never reached

```
1  def forever(n):
2      return forever(n + 1)      # no base case at all
3
4  forever(0)
5  # RecursionError: maximum recursion depth exceeded
```

2 Every call makes another. Python stops the program at about 1000 calls deep.

A base case that is skipped does the same: if n == 0 with n going 5, 3, 1, -1 … is never reached.

Recursion with more than one branch

```
1  def fib(n):
2      if n < 2:
3          return n
4      return fib(n - 1) + fib(n - 2)
5
6  print([fib(i) for i in range(7)])   # [0, 1, 1, 2, 3, 5, 8]
```

2-3 Two base cases in one: fib(0) is 0, fib(1) is 1.

4 Each call makes two more. Exam questions keep n small enough to trace.

What to know about recursion

- Every recursive function could be written as a loop, and the reverse.
- Recursion suits problems made of smaller copies of themselves.
- Each waiting call uses memory. Deep recursion hits the limit: RecursionError.
- To trace: list the calls going down, then the returns coming up.

yield hands out one value, then pauses

```
1  def count_up(n):
2      i = 0
3      while i < n:
4          yield i             # hand out i, then pause here
5          i += 1
6  print(list(count_up(3)))     # [0, 1, 2]
7  for x in count_up(2):
8      print(x, end=" ")        # 0 1
```

4 Any function containing yield is a generator function.

6-8 list() and for loops ask for values one after another until there are none left.

Calling a generator function runs nothing yet

```
1  def gen():
2      print("started")
3      yield 1
4      yield 2
5  g = gen()                  # nothing is printed yet
6  print(type(g))             # <class 'generator'>
7  print(next(g))             # started, then 1
8  print(list(g))             # [2]: only what is left
9  print(list(g))             # []: used up
```

5 The call just creates a generator object. The body has not started.

7 next() asks for one value: the body runs up to the first yield.

8-9 A generator can be used up only once. After that it gives nothing.

next() on a used-up generator raises StopIteration.

Two ways a function can hand back values

	RETURN	YIELD
What the call gives	the returned value	a generator object
Values handed back	one	as many as the yields reached
After handing back	the function is finished	paused, resumes on request
Reused	every call runs it again	used up after one pass

ON THE EXAM

A function containing yield always returns a generator, even if the yield is never reached. A return inside it simply ends it.

Tracing a generator is tracing a loop

```
 1  def every_other(data):
 2      for d in data[::2]:
 3          yield d
 4  for x in every_other("abcdef"):
 5      print(x, end="")          # ace
 6  print()
 7  def stars(n):
 8      s = "*"
 9      for i in range(n):
10          yield s
11          s += s
12  print(list(stars(3)))        # ['*', '**', '****']
```

2-3 The slice takes every second character: a, c, e. One yield per character.

10-11 Each value is yielded before s doubles, so the first value is a single star.

What to carry into the exam

- Recursion needs a **base case that is reached**. Otherwise: RecursionError.
- Trace calls going down, then returns coming up. A base value of 0 zeroes a product.
- Work before the recursive call happens on the way down; after it, on the way up.
- A function with **yield** returns a generator; its body runs only when values are requested.
- A generator can be used up only once. The order of yield and update decides every value.

What is the output of this program?

```
1  def walk(n):
2      if n == 1:
3          return 1
4      return n + walk(n - 2)
5
6  print(walk(5))
```

A 15

B 9

C 8

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def walk(n):
2      if n == 1:
3          return 1
4      return n + walk(n - 2)
5
6  print(walk(5))
```

walk(5) = 5 + walk(3) = 5 + 3 + walk(1) = 5 + 3 + 1 = 9.

A 15

Adds every number from 5 down to 1. The step is 2.

B 9

5, then 3, then the base case 1: 5 + 3 + 1.

C 8

Leaves out the base case's 1.

D The code raises a RecursionError

From 5, stepping by 2, n does land on 1.

THE SIGNAL

Check the base case is actually reached by the steps the recursion takes.

What happens when this program runs?

```
1  def walk(n):
2      if n == 0:
3          return 0
4      return n + walk(n - 2)
5
6  print(walk(5))
```

A 9 is printed

B 0 is printed

C 6 is printed

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  def walk(n):
2      if n == 0:
3          return 0
4      return n + walk(n - 2)
5
6  print(walk(5))
```

n goes 5, 3, 1, -1, -3 … It is always odd, so it never equals 0.

A 9 is printed

Assumes the recursion stops at 1. The base case is 0.

B 0 is printed

The base case is never reached, so 0 is never returned.

C 6 is printed

Treats the step as 1. It is 2.

D The code raises a RecursionError

Odd n minus 2 is always odd: n skips 0 for ever.

THE SIGNAL

A base case tested with == can be jumped over. Check the steps can land on it exactly.

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          show(n - 1)
4          print(n, end="")
5
6  show(3)
```

A 321

B 0123

C 123

D 3210

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def show(n):
2      if n > 0:
3          show(n - 1)
4          print(n, end="")
5
6  show(3)
```

The print comes after the recursive call, so it happens on the way back up: 1, then 2, then 3.

A 321

That order needs the print before the recursive call.

B 0123

show(0) prints nothing: its if is false.

C 123

Each call prints after the deeper call returns, so 1 comes first.

D 3210

Wrong order, and includes 0, which is never printed.

THE SIGNAL

Work after the recursive call happens as the calls unwind, deepest first.

What is the output of this program?

```
1  def mul(n):
2      if n == 0:
3          return 0
4      return n * mul(n - 1)
5
6  print(mul(4))
```

A 24

B 0

C 10

D The code raises a RecursionError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def mul(n):
2      if n == 0:
3          return 0
4      return n * mul(n - 1)
5
6  print(mul(4))
```

4 * 3 * 2 * 1 * 0 = 0.

A 24

What a base case of 1 would give. This one returns 0.

B 0

The base case multiplies everything by 0.

C 10

Adds instead of multiplying.

D The code raises a RecursionError

n steps down by 1, so it does reach 0.

THE SIGNAL

A product's base case must return 1. Returning 0 makes every result 0.

What is the output of this program?

```
1  def gen():
2      yield 1
3      yield 2
4      yield 3
5
6  g = gen()
7  print(next(g), list(g), list(g))
```

A `1 [2, 3] []`

B `1 [1, 2, 3] [1, 2, 3]`

C `1 [2, 3] [2, 3]`

D `[1] [2] [3]`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def gen():
2      yield 1
3      yield 2
4      yield 3
5
6  g = gen()
7  print(next(g), list(g), list(g))
```

One generator, used from left to right: next takes 1, list takes the rest, then nothing is left.

A `1 [2, 3] []`

next takes 1; the first list takes the rest; the generator is then used up.

B `1 [1, 2, 3] [1, 2, 3]`

Treats each list(g) as a fresh start. g is one generator.

C `1 [2, 3] [2, 3]`

Forgets that a generator can be used up only once.

D `[1] [2] [3]`

next gives a value, not a list.

THE SIGNAL

A generator remembers its position and is used up after one pass. Calling gen() again would start a new one.

What is the output of this program?

```
1  def grow(n):
2      s = "a"
3      for _ in range(n):
4          yield s
5          s = s + "b"
6
7  for x in grow(3):
8      print(x, end=" ")
```

A abb

B a ab abb abbb

C ab abb abbb

D a ab abb

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def grow(n):
2      s = "a"
3      for _ in range(n):
4          yield s
5          s = s + "b"
6
7  for x in grow(3):
8      print(x, end=" ")
```

Three passes. Each yields s first, then adds a b: a, ab, abb.

A `abb`

Only the last value. The loop prints every value yielded.

B `a ab abb abbb`

Four values from three passes. range(3) gives three yields.

C `ab abb abbb`

Updates before yielding. Here the yield comes first.

D `a ab abb`

Three yields, each before its update.

THE SIGNAL

Count the yields, then check whether each comes before or after the update.